

How I Used Hindsight to Detect When Engineering Decisions Drift

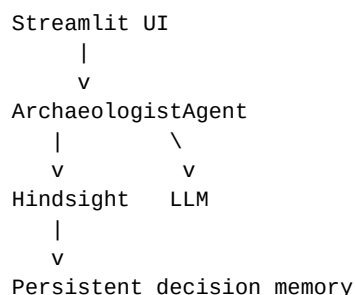
A line like `RATE_LIMIT = 100` is trivial to read and almost impossible to audit. The syntax tells you everything about what the code does right now. It tells you nothing about why someone set it to 100 instead of 1000, whether that reason still holds, or whether the person who could answer that question left the company two years ago.

I built Archaeologist to close that gap: a system that remembers *why* an engineering decision was made, not just what it currently is, and that can tell you when the reasoning behind an old decision has quietly gone stale.

What it does and how the pieces fit

Archaeologist has three operations that matter. An engineer records a decision — an entity like `search_api.rate_limit`, what was decided, why, who decided it, and when. Later, a different engineer asks "why does this exist?" and gets an answer built from what was actually recorded, with a timeline and a cautious flag if the original reasoning looks like it may no longer apply.

The application layer is small on purpose:



The `ArchaeologistAgent` is the only thing that talks to both sides. A dedicated memory module is the *only* file in the codebase that imports the [Hindsight](#) client — everything else works against plain Python models. There's no separate application database sitting next to Hindsight, no vector store to keep in sync, no cache that can drift out from under the source of truth. Hindsight *is* the persistence layer. I didn't build a memory system and bolt Hindsight underneath it; I built the product against Hindsight's [RETAIN and RECALL primitives](#) directly, and the shape of those two operations ended up shaping most of the interesting decisions in this codebase.

The core story: making history additive, not overwritable

The single design decision that mattered most was refusing to let an "update" ever destroy an "original."

Hindsight documents are addressed by a `document_id`, and if you retain content under a `document_id` that already exists, Hindsight replaces the prior document. That's the correct default for most memory use cases — you usually want the latest state. It is exactly the wrong default for a system whose entire job is remembering what used to be true and who said so.

So every write mints a fresh ID:

```
# app/memory/hindsight_store.py

# Unique per record: reusing a document_id would delete the earlier document
# and with it the history this product exists to protect.
document_id = (
    f"{normalize_entity(record.entity)}:{record.kind.value}:{uuid.uuid4().hex[:12]}"
)
```

That one line is the difference between a system that tracks *current state* and a system that tracks *history*. An update to `search_api.rate_limit` doesn't touch the document Priya wrote in 2023. It's a new document, tagged with the same entity, that Hindsight's retrieval will later surface alongside the original.

The second decision follows from the first: once history is additive, something has to reconstruct the order of events, and I decided early on that the something would never be the language model. Chronology is a sorting problem, and sorting problems don't need an LLM:

```
# app/agent/timeline.py

def _sort_key(memory: RecalledMemory) -> tuple[int, datetime]:
    """Order oldest-first, with undated memories pushed to the end."""
    moment = memory.effective_time
    if moment is None:
        return (1, datetime.max.replace(tzinfo=timezone.utc))
    normalized = moment if moment.tzinfo else moment.replace(tzinfo=timezone.utc)
    return (0, normalized)

def sort_memories_chronologically(memories: list[RecalledMemory]) -> list[RecalledMemory]:
    """Return memories oldest-first. Hindsight returns them by relevance."""
    return sorted(memories, key=_sort_key)
```

`effective_time` prefers the decision's real-world date over the date it happened to be typed into the system — which matters, because I also pass that real-world timestamp into Hindsight's `retain()` call rather than letting it default to "now." If you let every write get stamped with ingestion time, you lose the ability to backfill history or correct a record after the fact without scrambling the timeline. Hindsight's temporal retrieval and my own `sort_memories_chronologically` need to agree on what "when" means, and the only way to guarantee that is to feed both of them the same date.

The model only ever sees a timeline that's already been built. It explains history; it doesn't get to decide what the history was.

Why RECALL isn't the last step

A question like "why is `search_api.rate_limit` still 100?" doesn't go straight to an LLM. It goes to Hindsight first — a broad semantic recall over the whole memory bank, plus, when the question names a specific entity, a second recall scoped to that entity's tag. Both result sets get merged before anything gets synthesized.

Once I have the memories, I treat them as data, not instructions. That distinction matters more than it sounds like it should: the text inside a recalled memory originated from whatever an engineer typed into a decision record, and an LLM asked to synthesize an answer from a pile of retrieved text will happily follow anything that looks like an instruction embedded in that text. So the synthesis prompt fences retrieved memories explicitly as untrusted content, and — this is the part I actually enforce in code rather than just asking nicely in a prompt — anything the model cites has to correspond to something that was really retrieved:

```
# app/agent/archaeologist.py

def _filter_ids(raw: Any, known_ids: set[str]) -> list[str]:
    """Keep only ids that were genuinely retrieved, preserving order.

    Guards against a model citing a memory that does not exist.
    """
    if not isinstance(raw, list):
        return []
    seen: list[str] = []
    for item in raw:
        candidate = str(item)
        if candidate in known_ids and candidate not in seen:
            seen.append(candidate)
    return seen
```

If the model hallucinates a memory ID — invents a citation that sounds plausible but doesn't correspond to anything Hindsight actually returned — it gets silently dropped before it ever reaches the UI. The model can summarize what's there. It cannot manufacture evidence that isn't.

The drift classification follows the same philosophy: bounded, explicit states instead of a free-text verdict.

```
# app/domain/models.py

class DriftStatus(str, Enum):
    """How confident we are that the original reasoning still holds."""

    NO_EVIDENCE = "no_evidence"
    LIKELY_CURRENT = "likely_current"
    NEEDS_REVIEW = "needs_review"
    MAY_BE_OUTDATED = "may_be_outdated"
```

Notice what's *not* in that enum: there's no `WRONG` or `BROKEN` state. Archaeologist never tells you the code is incorrect — it doesn't have the authority to make that claim, and honestly neither does an LLM reading a handful of retrieved facts. It tells you the *record* suggests the original reasoning may no longer apply, which is a claim about the historical evidence, not about your production system.

What this actually looks like end to end

Here's the sequence that exercises every piece of this: In April 2023, Priya Raman records that `search_api.rate_limit` is 100 requests/sec because Elasticsearch became unstable above that threshold. In August 2024, Marcus Webb records that the cluster was upgraded and the instability that motivated the original limit is gone — the limit itself wasn't touched.

Ask "why is `search_api.rate_limit` still 100?" before either of those exists, and you get an honest, unglamorous answer: *"I don't have recorded historical context for this decision."* No guessing, no plausible-sounding fabrication. That cold-start state is not a lesser version of the feature — it's the thing that proves the system is actually grounded in memory rather than producing confident-sounding text regardless of what it knows.

After both memories are retained, asking the same question again recalls both, reconstructs the timeline in the order they actually happened, and synthesizes an answer that names both engineers and both dates — then classifies the drift status as `may_be_outdated`, because the record shows the original constraint was resolved without the limit ever being revisited. Running that full round trip — two retains, a recall, and a synthesis call — against Hindsight Cloud and Groq end to end takes well under twenty seconds on my machine. I'm not claiming that as a benchmark; it's just fast enough that "ask why, get a grounded answer" doesn't feel like it's fighting the tooling.

Not every bug was in the interesting part of the system, either. The most annoying failure I hit had nothing to do with Hindsight or the LLM — it was a `UnicodeEncodeError` on Windows because the console defaults to `cp1252` and a Groq completion legitimately contained a narrow no-break space that codepage can't encode. `RETAIN` worked, `RECALL` worked, synthesis worked, and the whole thing still crashed on a `print()` call. Worth remembering that the parts of a system most likely to break in practice aren't always the parts you spent the most time designing.

What I'd tell someone building something similar

Model history as its own type, not a text blob. Once a decision had an explicit entity, reason, author, and date instead of being buried in free text, everything downstream — retrieval scoping, timeline construction, drift comparison — got dramatically simpler to reason about.

Don't hand the LLM anything that has a correct answer computable in code. Chronology, citation validity, and whether a memory was actually retrieved are all things I can verify deterministically. The model's job is explaining evidence, not producing it.

Persistent memory is a product decision, not an infrastructure swap. RETAIN is what makes history accumulate instead of getting silently overwritten by the next update. RECALL is what makes that accumulated history reachable months later. Take persistence away and there's no timeline to reconstruct and nothing for drift detection to compare against — the feature doesn't degrade gracefully, it stops existing.

"I don't know" has to be a real, reachable state. A system that always sounds confident is indistinguishable, from the outside, from one that's making things up. The cold-start path where nothing has been recorded yet is what makes every other answer credible.

Surfacing conflicting memories beats silently picking one. When two recorded decisions disagree, the honest move is showing both, with their authors and dates, and saying so — not letting the model quietly decide which engineer to believe.

None of this is about building [a smarter model](#) — it's about giving a fairly ordinary model a bounded, trustworthy record to reason over, and being deliberate about which parts of the answer get to come from the model at all. That turned out to be the actual engineering problem.